

3x3 Weight-Stationary Systolic Array Data Flow

1. What This RTL Is Actually Doing

This note matches the **weight-stationary** RTL in:

- `weightStationaryVariant/systolicArrayWeightStationary.sv`
- `weightStationaryVariant/multiplierBlockWeightStationary.sv`
- `weightStationaryVariant/weightLoader.sv`
- `weightStationaryVariant/dataOrchestratorWeightStationary.sv`

The dataflow has two phases:

1. **Weight-load phase:** weights are pushed in from the top while `writeEnable = 1`. Each PE stores one weight in `weightReg`.
2. **Compute phase:** activations enter from the left while `writeEnable = 0`. Each PE multiplies the activation by its stored weight, then adds into the vertical partial sum flowing downward.

So this is not output-stationary. The stationary quantity is the **weight**.

2. PE-Level Operation

Each PE implements:

$$\text{bottomOut} \leftarrow \text{topIn} + \text{leftIn} \cdot \text{weightReg}$$

and

$$\text{rightOut} \leftarrow \text{leftIn}.$$

That means:

- activations move left to right,
- partial sums move top to bottom,
- weights stay stored inside the PEs after the load phase.

3. Matrix Mapping

Let

$$A = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix}, \quad B = \begin{bmatrix} b_{00} & b_{01} & b_{02} \\ b_{10} & b_{11} & b_{12} \\ b_{20} & b_{21} & b_{22} \end{bmatrix}.$$

In this RTL:

- PE(i, j) stores the weight b_{ij} after the weight-load phase,
- row stream i carries column i of A over time:

$$a_{0i}, a_{1i}, a_{2i}.$$

At stream step k , the value currently entering row i is a_{ki} . When that wavefront passes through column j , the column computes

$$\sum_{i=0}^2 a_{ki} b_{ij}.$$

That is exactly

$$C_{kj} = \sum_{i=0}^2 a_{ki} b_{ij},$$

which is standard matrix multiplication:

$$C = AB.$$

4. Why Column Streams of A Still Produce Rows of C

This is the subtle point.

Although row 0, row 1, and row 2 receive:

$$\text{column 0 of } A, \quad \text{column 1 of } A, \quad \text{column 2 of } A,$$

they do *not* represent three different output rows.

Instead, at stream index k , the three injected values are

$$a_{k0}, a_{k1}, a_{k2},$$

which together form **row k of A** .

So one time step of the left-side injection reconstructs one full row of A across the array rows. The fixed PE weights then dot that row with each column of B .

5. Explicit 3x3 PE Equations

The stored weights are:

$$\begin{array}{lll} \text{PE}(0, 0) : b_{00} & \text{PE}(0, 1) : b_{01} & \text{PE}(0, 2) : b_{02} \\ \text{PE}(1, 0) : b_{10} & \text{PE}(1, 1) : b_{11} & \text{PE}(1, 2) : b_{12} \\ \text{PE}(2, 0) : b_{20} & \text{PE}(2, 1) : b_{21} & \text{PE}(2, 2) : b_{22} \end{array}$$

Then the bottom outputs of the three columns produce:

$$C_{k0} = a_{k0}b_{00} + a_{k1}b_{10} + a_{k2}b_{20},$$

$$C_{k1} = a_{k0}b_{01} + a_{k1}b_{11} + a_{k2}b_{21},$$

$$C_{k2} = a_{k0}b_{02} + a_{k1}b_{12} + a_{k2}b_{22},$$

for $k = 0, 1, 2$.

Expanding all nine outputs:

$$C_{00} = a_{00}b_{00} + a_{01}b_{10} + a_{02}b_{20},$$

$$C_{01} = a_{00}b_{01} + a_{01}b_{11} + a_{02}b_{21},$$

$$C_{02} = a_{00}b_{02} + a_{01}b_{12} + a_{02}b_{22},$$

$$C_{10} = a_{10}b_{00} + a_{11}b_{10} + a_{12}b_{20},$$

$$C_{11} = a_{10}b_{01} + a_{11}b_{11} + a_{12}b_{21},$$

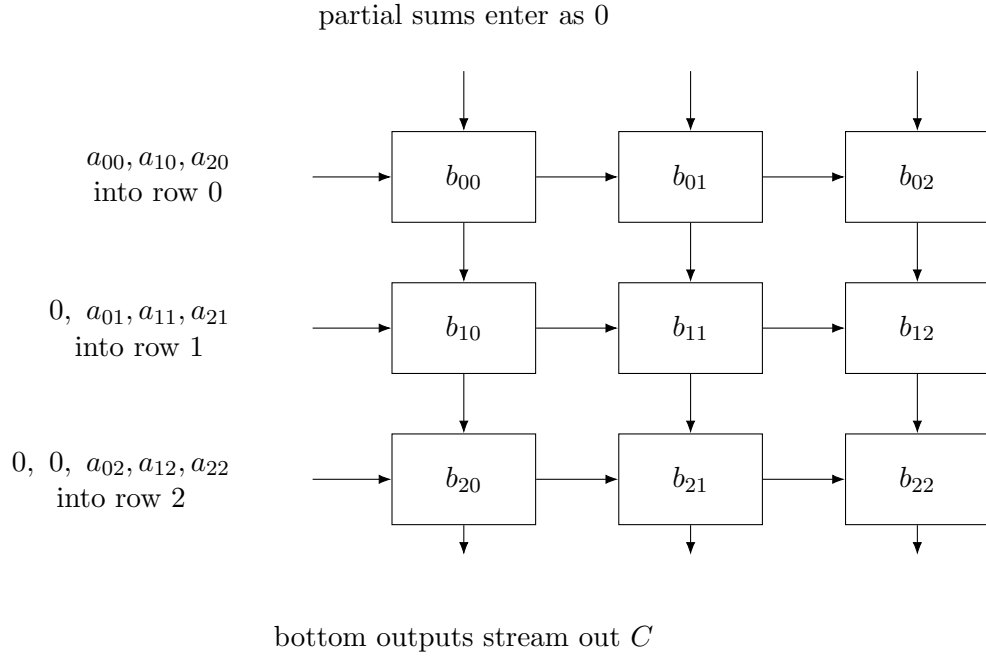
$$C_{12} = a_{10}b_{02} + a_{11}b_{12} + a_{12}b_{22},$$

$$C_{20} = a_{20}b_{00} + a_{21}b_{10} + a_{22}b_{20},$$

$$C_{21} = a_{20}b_{01} + a_{21}b_{11} + a_{22}b_{21},$$

$$C_{22} = a_{20}b_{02} + a_{21}b_{12} + a_{22}b_{22}.$$

6. Structural Picture



The labels inside the PEs are the weights that remain stationary during computation.

7. Timing: Weight Loading

The weight loader runs first while `writeEnable = 1`. During this phase:

- top inputs carry weight values,
- left inputs are effectively zero,
- each PE captures `topIn[WIDTH-1:0]` into `weightReg`.

The weights are sent *downward* through the array during loading, so each column gets filled from top to bottom over several cycles.

Conceptually, column j is loaded so that:

$$\begin{aligned} \text{PE}(0, j) &\leftarrow b_{0j} \\ \text{PE}(1, j) &\leftarrow b_{1j} \\ \text{PE}(2, j) &\leftarrow b_{2j} \end{aligned}$$

once the load phase finishes.

8. Timing: Compute Phase and Stagger

After the weights are loaded, `writeEnable` goes low and the activation stream begins.

The orchestrator staggers the rows:

$$\begin{aligned} \text{row 0: } &a_{00}, a_{10}, a_{20} \\ \text{row 1: } &0, a_{01}, a_{11}, a_{21} \\ \text{row 2: } &0, 0, a_{02}, a_{12}, a_{22} \end{aligned}$$

So yes: the row below arrives one cycle later, and the next row one more cycle later. Equivalently, the value a_{ki} is injected into row i at time

$$t = t_0 + i + k,$$

where t_0 is the start of the compute phase.

Then it takes one more cycle per PE to move to the right, so a_{ki} reaches $\text{PE}(i, j)$ at

$$t = t_0 + i + j + k.$$

That is the diagonal wavefront.

9. One Full Example: How C_{12} Is Formed

Take column $j = 2$ and output row $k = 1$.

We want:

$$C_{12} = a_{10}b_{02} + a_{11}b_{12} + a_{12}b_{22}.$$

During the compute phase:

- row 0 contributes a_{10} and $\text{PE}(0, 2)$ multiplies by b_{02} ,
- row 1 contributes a_{11} and $\text{PE}(1, 2)$ multiplies by b_{12} ,
- row 2 contributes a_{12} and $\text{PE}(2, 2)$ multiplies by b_{22} .

These three products are accumulated down column 2 as the partial sum travels downward, and the finished value exits the bottom of that column.

10. What the Bottom Outputs Mean

The array output in `systolicArrayWeightStationary.sv` is:

$$\text{result}[j] = \text{verticalData}[N][j].$$

So each bottom output corresponds to one output column:

- bottom output 0 streams values for column 0 of C ,
- bottom output 1 streams values for column 1 of C ,
- bottom output 2 streams values for column 2 of C .

Because of the skew, the values do not all appear at once. They emerge over time in a staggered pattern.

11. Short Summary

- This design is weight-stationary because each PE stores one weight b_{ij} .
- The left-going streams are staggered activation values from A .
- At stream step k , the three row injections form row k of A across the array rows.
- Each column of PEs computes one dot product with a column of B .
- The bottom outputs therefore stream the entries of $C = AB$.